EXPT NO: 9 DATE: 01-11-2026	TEXT CLASSIFICATION AND TEXT GENERATION USING TRANSFORMERS NAME: Jeevanantham K ROLL NO: 24BAD047

PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What is a Transformer model in Natural Language Processing (NLP)?

Ans: A Transformer is a neural network architecture based on the self-attention mechanism. It processes all tokens of a sequence in parallel and learns how strongly each token relates to every other token, producing context-aware representations. It is built from encoder and/or decoder stacks (multi-head attention, feed-forward layers, residual connections, layer normalisation, positional encodings) and replaced RNN/LSTM models because it trains faster and captures long-range dependencies better.

2. What is BERT, and how is it used for text classification?

Ans: BERT (Bidirectional Encoder Representations from Transformers) is an encoder-only Transformer pretrained on large corpora with Masked Language Modelling and Next Sentence Prediction, so it understands context from both directions. For classification the input starts with a [CLS] token; the final hidden state of [CLS] is passed to a linear layer that outputs class probabilities.

3. What is fine-tuning in the context of pretrained language models?

Ans: Fine-tuning is continuing the training of a pretrained model on a smaller, task-specific labelled dataset, usually with a small learning rate, so its general language knowledge is adapted to the target task. It needs far less data and compute than training from scratch.

4. What is the difference between text classification and text generation?

Ans: Text classification maps an input text to one label from a fixed set of classes (e.g. positive or negative). Text generation produces new text token by token (e.g. a summary or translation); its output is open-ended and of variable length.

5. What is a sequence-to-sequence model, and how is it used in text generation?

Ans: A sequence-to-sequence model has an encoder that converts the input sequence into contextual representations and a decoder that generates the output sequence autoregressively while attending to the encoder output. Examples are T5 and BART; they are used for summarization, translation, question answering and paraphrasing.

AIM

To implement text classification by fine-tuning a pretrained BERT model and develop a simple sequence-to-sequence text generation application using Transformer models.

SOFTWARE REQUIREMENTS

- Python
- Google Colab
- Hugging Face Transformers
- Hugging Face Datasets
- PyTorch
- NumPy
- Pandas
- Matplotlib

Dataset used: IMDb Movie Reviews (stanfordnlp/imdb), 25,000 train and 25,000 test reviews with binary sentiment labels; a random subset of 4,000 training and 1,000 testing reviews was used. Models: BERT Base Uncased (google-bert/bert-base-uncased) for classification and T5-small (google-t5/t5-small) for sequence-to-sequence generation.

OBJECTIVES

Students will be able to:

- Understand the architecture and working of Transformer models in NLP.

- Understand the concepts of text classification and text generation.
- Load and preprocess text datasets for NLP applications.
- Fine-tune a pretrained BERT model for text classification.
- Evaluate the classification model using appropriate performance metrics.
- Implement a sequence-to-sequence application using a pretrained Transformer model.
- Generate text using the model's text generation capabilities.
- Analyze and compare the outputs produced by the Transformer models.

PROCEDURE

PART A: DATASET PREPARATION AND TEXT PREPROCESSING

Perform the following tasks:

- Load a text dataset such as the IMDb movie review dataset.
- Explore the dataset and examine sample text entries and their labels.
- Check the number of samples and the distribution of class labels.
- Separate the dataset into training and testing sets or use the predefined splits.
- Load a pretrained tokenizer corresponding to the BERT model.
- Tokenize the text using the tokenizer.
- Apply padding and truncation to maintain consistent input lengths.
- Convert the tokenized data into the required format for model training.
- Prepare the dataset for batch-wise processing.

A1. Library installation — Code

```
!pip install -q transformers datasets accelerate scikit-learn
```

A2. Student details and device setup — Code

```
import os, time, random, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
warnings.filterwarnings("ignore")


STUDENT_NAME = "Jeevanantham K"
ROLL_NO      = "24BAD047"
DATE         = "01-11-2026"


SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Student : {STUDENT_NAME} | Roll No : {ROLL_NO} | Date : {DATE}")
print("Device  :", device, "|", torch.cuda.get_device_name(0) if device.type == "cuda" else "CPU (training will be slow; switch to GPU)")
```

A2. Student details and device setup — Output

```
Student : Jeevanantham K | Roll No : 24BAD047 | Date : 01-11-2026
Device  : cuda | Tesla T4
```

A3. Dataset loading — Code

```
from datasets import load_dataset

print(f"[Roll No: {ROLL_NO} | {STUDENT_NAME}] PART A - Dataset loading")
dataset = load_dataset("stanfordnlp/imdb")
print(dataset)

label_names = {0: "NEGATIVE", 1: "POSITIVE"}

print("\nNumber of samples -> train:", len(dataset["train"]), "| test:", len(dataset["test"]))
```

A3. Dataset loading — Output

```
[Roll No: 24BAD047 | Jeevanantham K] PART A - Dataset loading
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
    })
})

Number of samples -> train: 25000 | test: 25000
```

A4. Sample entries and labels — Code

```python
# Explore sample entries and their labels
for i in [0, 1, 12500, 12501]:
    row = dataset["train"][i]
    print(f"Index {i} | Label = {row['label']} ({label_names[row['label']]})")
    print(row["text"][:300], "...\n")
```

A4. Sample entries and labels — Output

```
Index 0 | Label = 0 (NEGATIVE)
I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was first released in 1967. I also heard that at first it was seized by U.S. customs if it ever tried  to enter this country, therefore being a fan of films considered "controversial" I really h ...

Index 1 | Label = 0 (NEGATIVE)
"I Am Curious: Yellow" is a risible and pretentious steaming pile. It doesn't matter what one's political views are because this film can hardly be taken seriously on any level. As for the claim that frontal male nudity is an automatic NC-17, that isn't true. I've seen R-rated films with male nudity ...

Index 12500 | Label = 1 (POSITIVE)
Zentropa has much in common with The Third Man, another noir-like film set among the rubble of postwar Europe. Like TTM, there is much inventive camera work. There is an innocent American who gets emotionally involved with a woman he doesn't really understand, and whose naivety is all the more strik ...

Index 12501 | Label = 1 (POSITIVE)
Zentropa is the most original movie I've seen in years. If you like unique thrillers that are influenced  by film noir, then this is just the right cure for all of those Hollywood summer blockbusters clogging the theaters these days. Von Trier's follow-ups like Breaking the Waves have gotten more acc ...
```

A5. Class distribution and review length — Code

```python
# Class label distribution
train_df = dataset["train"].to_pandas()
test_df  = dataset["test"].to_pandas()

print("Train distribution:\n", train_df["label"].value_counts().rename(label_names))
print("\nTest distribution:\n", test_df["label"].value_counts().rename(label_names))

train_df["word_count"] = train_df["text"].str.split().str.len()
print("\nReview length (words):")
print(train_df["word_count"].describe())

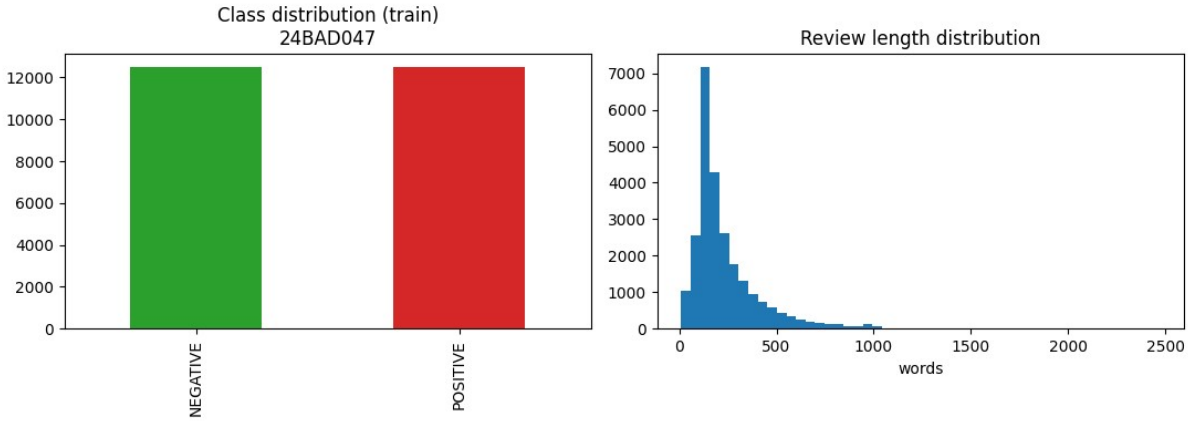
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
train_df["label"].map(label_names).value_counts().plot(kind="bar", ax=ax[0], color=["tab:green", "tab:red"])
ax[0].set_title(f"Class distribution (train)\n{ROLL_NO}"); ax[0].set_xlabel("")
ax[1].hist(train_df["word_count"], bins=50, color="tab:blue")
ax[1].set_title("Review length distribution"); ax[1].set_xlabel("words")
plt.tight_layout(); plt.show()
```

A5. Class distribution and review length — Output

```
Train distribution:
 label
NEGATIVE    12500
POSITIVE    12500
Name: count, dtype: int64

Test distribution:
 label
NEGATIVE    12500
POSITIVE    12500
Name: count, dtype: int64

Review length (words):
count    25000.000000
mean       233.787200
std        173.733032
min         10.000000
25%        127.000000
50%        174.000000
75%        284.000000
max       2470.000000
Name: word_count, dtype: float64
```



A6. Train and test subset — Code

```python
# Use predefined splits; take a balanced random subset for fast training
TRAIN_SIZE, TEST_SIZE = 4000, 1000     # increase for better accuracy (e.g. 25000 / 25000)
train_ds = dataset["train"].shuffle(seed=SEED).select(range(TRAIN_SIZE))
test_ds  = dataset["test"].shuffle(seed=SEED).select(range(TEST_SIZE))
print("Subset sizes -> train:", len(train_ds), "| test:", len(test_ds))
print("Subset train labels:", pd.Series(train_ds["label"]).value_counts().to_dict())
```

A6. Train and test subset — Output

```
Subset sizes -> train: 4000 | test: 1000
Subset train labels: {0: 2015, 1: 1985}
```

A7. Tokenizer — Code

```python
from transformers import AutoTokenizer

MODEL_NAME = "bert-base-uncased"
MAX_LEN = 256
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

# Tokenizer demonstration
sample = "This movie was absolutely fantastic!"
enc = tokenizer(sample)
print("Text      :", sample)
print("Tokens    :", tokenizer.convert_ids_to_tokens(enc["input_ids"]))
print("Input IDs :", enc["input_ids"])
print("Attention :", enc["attention_mask"])
```

A7. Tokenizer — Output

```
Text      : This movie was absolutely fantastic!
Tokens    : ['[CLS]', 'this', 'movie', 'was', 'absolutely', 'fantastic', '!', '[SEP]']
Input IDs : [101, 2023, 3185, 2001, 7078, 10392, 999, 102]
Attention : [1, 1, 1, 1, 1, 1, 1, 1]
```

A8. Tokenization with padding and truncation — Code

```python
def tokenize_fn(batch):
    return tokenizer(batch["text"], padding="max_length", truncation=True, max_length=MAX_LEN)

tokenized_train = train_ds.map(tokenize_fn, batched=True)
tokenized_test  = test_ds.map(tokenize_fn, batched=True)

# Convert to PyTorch format (Trainer expects the column name 'labels')
tokenized_train = tokenized_train.rename_column("label", "labels").remove_columns(["text"])
tokenized_test  = tokenized_test.rename_column("label", "labels").remove_columns(["text"])
tokenized_train.set_format("torch")
tokenized_test.set_format("torch")

print(f"[Roll No: {ROLL_NO}] Tokenized columns:", tokenized_train.column_names)
print("Example shapes:", {k: tuple(v.shape) for k, v in tokenized_train[0].items()})
```

A8. Tokenization with padding and truncation — Output

```
[Roll No: 24BAD047] Tokenized columns: ['labels', 'input_ids', 'token_type_ids', 'attention_mask']
Example shapes: {'labels': (), 'input_ids': (256,), 'token_type_ids': (256,), 'attention_mask': (256,)}
```

A9. Batch-wise processing — Code

```python
# Batch-wise processing with DataLoader
from torch.utils.data import DataLoader

loader = DataLoader(tokenized_train, batch_size=16, shuffle=True)
batch = next(iter(loader))
print(f"[Roll No: {ROLL_NO} | {STUDENT_NAME}] One batch:")
for k, v in batch.items():
    print(f"  {k:15s} shape = {tuple(v.shape)}")
```

A9. Batch-wise processing — Output

```
[Roll No: 24BAD047 | Jeevanantham K] One batch:
  labels          shape = (16,)
  input_ids       shape = (16, 256)
  token_type_ids  shape = (16, 256)
  attention_mask  shape = (16, 256)
```

PART B: FINE-TUNING BERT FOR TEXT CLASSIFICATION

Perform the following tasks:

- Load a pretrained BERT model for sequence classification.
- Configure the model for the required number of classes.
- Prepare the tokenized training and testing datasets.
- Define the training parameters, including learning rate, batch size, and number of epochs.
- Fine-tune the BERT model using the training dataset.
- Evaluate the trained model using the testing dataset.
- Calculate appropriate performance metrics, such as accuracy, precision, recall, and F1-score.
- Test the model using new text inputs.
- Display the predicted class labels and corresponding evaluation results.
- Save the fine-tuned model and tokenizer.

B1. Model loading — Code

```python
from transformers import AutoModelForSequenceClassification, TrainingArguments, Trainer
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

print(f"[Roll No: {ROLL_NO} | {STUDENT_NAME}] PART B - Model loading")
id2label = {0: "NEGATIVE", 1: "POSITIVE"}
label2id = {"NEGATIVE": 0, "POSITIVE": 1}

model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME, num_labels=2, id2label=id2label, label2id=label2id
)
print("Trainable parameters: {:,}".format(sum(p.numel() for p in model.parameters() if p.requires_grad)))
```

B1. Model loading — Output

```
[Roll No: 24BAD047 | Jeevanantham K] PART B - Model loading
Trainable parameters: 109,483,778
```

B2. Metrics and training parameters — Code

```python
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average="binary", zero_division=0)
    return {"accuracy": accuracy_score(labels, preds), "precision": precision, "recall": recall, "f1": f1}

# Training parameters
LEARNING_RATE = 2e-5
BATCH_SIZE    = 16
EPOCHS        = 2

training_args = TrainingArguments(
    output_dir="./bert_imdb_results",
    learning_rate=LEARNING_RATE,
    per_device_train_batch_size=BATCH_SIZE,
    per_device_eval_batch_size=32,
    num_train_epochs=EPOCHS,
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="no",
    logging_steps=25,
    fp16=torch.cuda.is_available(),
    report_to="none",
    seed=SEED,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_test,
    compute_metrics=compute_metrics,
)
print(f"Learning rate={LEARNING_RATE}, batch size={BATCH_SIZE}, epochs={EPOCHS}")
```

B2. Metrics and training parameters — Output

```
Learning rate=2e-05, batch size=16, epochs=2
```

B3. Fine-tuning — Code

```python
print(f"[Roll No: {ROLL_NO} | {STUDENT_NAME}] Fine-tuning started...")
t0 = time.time()
train_output = trainer.train()
print(f"Training finished in {(time.time()-t0)/60:.1f} min | final training loss = {train_output.training_loss:.4f}")
```

B3. Fine-tuning — Output

```
[Roll No: 24BAD047 | Jeevanantham K] Fine-tuning started...

 Epoch  Training Loss  Validation Loss  Accuracy  Precision   Recall       F1
     1       0.368495         0.272240     0.888   0.880567 0.891393 0.885947
     2       0.194983         0.292159     0.895   0.882236 0.905738 0.893832
Training finished in 1.8 min | final training loss = 0.2848
```

B4. Training loss curve — Code

```python
# Training loss curve
logs = pd.DataFrame(trainer.state.log_history)
loss_logs = logs.dropna(subset=["loss"])
plt.figure(figsize=(6, 3.5))
plt.plot(loss_logs["step"], loss_logs["loss"], marker="o")
plt.title(f"Training loss | {ROLL_NO} {STUDENT_NAME}")
plt.xlabel("step"); plt.ylabel("loss"); plt.grid(alpha=.3); plt.show()
```

B4. Training loss curve — Output



B5. Evaluation on test set — Code

```python
# Evaluation on the test set
print(f"[Roll No: {ROLL_NO} | {STUDENT_NAME}] Evaluation on test set")
pred_output = trainer.predict(tokenized_test)
y_true = pred_output.label_ids
y_pred = np.argmax(pred_output.predictions, axis=-1)

from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay

acc = accuracy_score(y_true, y_pred)
prec, rec, f1, _ = precision_recall_fscore_support(y_true, y_pred, average="binary")
print(f"Accuracy : {acc:.4f}\nPrecision: {prec:.4f}\nRecall   : {rec:.4f}\nF1-score : {f1:.4f}\n")
print(classification_report(y_true, y_pred, target_names=["NEGATIVE", "POSITIVE"]))

cm = confusion_matrix(y_true, y_pred)
ConfusionMatrixDisplay(cm, display_labels=["NEGATIVE", "POSITIVE"]).plot(cmap="Blues")
plt.title(f"Confusion matrix | {ROLL_NO}"); plt.show()
```

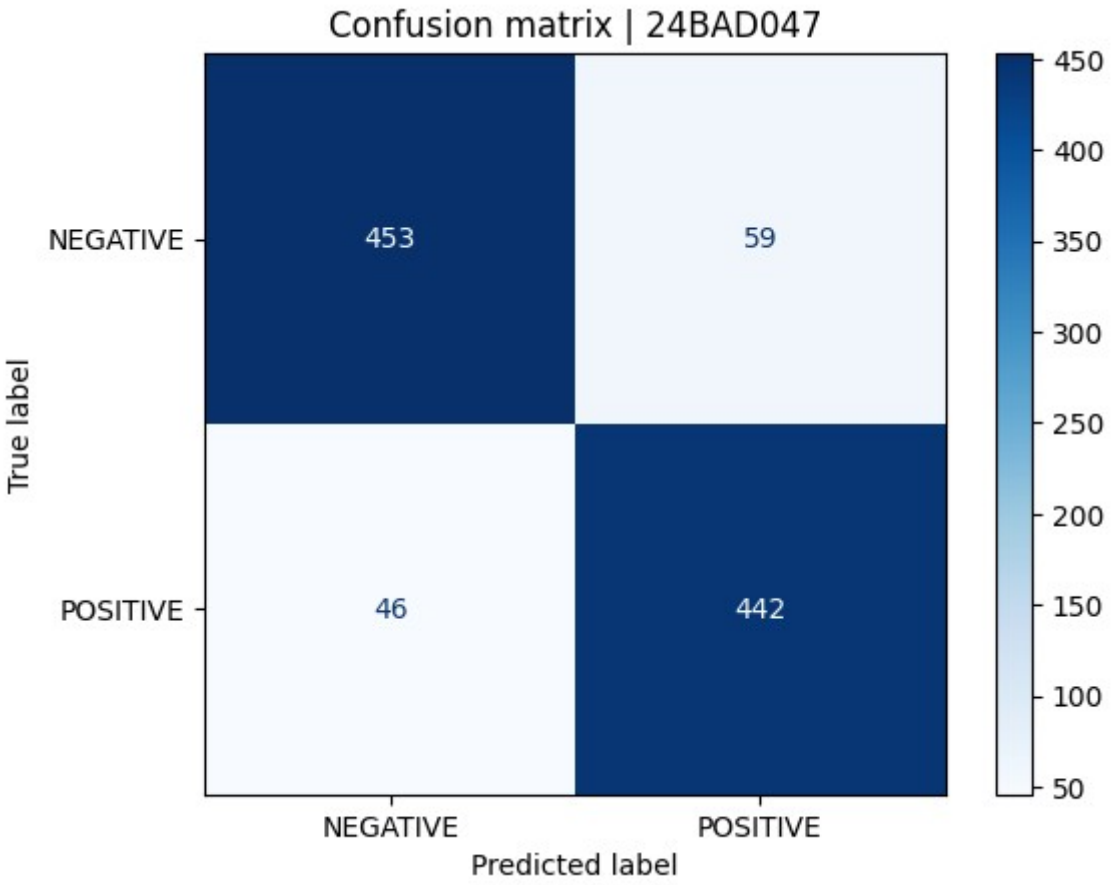
B5. Evaluation on test set — Output

```
[Roll No: 24BAD047 | Jeevanantham K] Evaluation on test set
Accuracy : 0.8950
Precision: 0.8822
Recall   : 0.9057
F1-score : 0.8938

              precision    recall  f1-score   support

    NEGATIVE       0.91      0.88      0.90       512
    POSITIVE       0.88      0.91      0.89       488

    accuracy                           0.90      1000
   macro avg       0.90      0.90      0.89      1000
weighted avg       0.90      0.90      0.90      1000
```



B6. Predictions on new text — Code

```python
# Testing the model with new text inputs
def predict_sentiment(texts):
    model.eval()
    enc = tokenizer(texts, padding=True, truncation=True, max_length=MAX_LEN, return_tensors="pt").to(model.device)
    with torch.no_grad():
        probs = torch.softmax(model(**enc).logits, dim=-1).cpu().numpy()
    return [(id2label[int(p.argmax())], float(p.max())) for p in probs]

new_texts = [
    "An outstanding film with brilliant acting and a gripping storyline.",
    "I loved every minute of it. Highly recommended!",
    "The plot was boring, the acting was terrible and I wasted my time.",
    "Worst movie I have seen this year. Absolutely awful.",
    "It was okay, some parts were good but the ending was disappointing.",
]
print(f"[Roll No: {ROLL_NO} | {STUDENT_NAME}] Predictions on new text")
for t, (lab, conf) in zip(new_texts, predict_sentiment(new_texts)):
    print(f"{lab:8s} ({conf:.2%}) <- {t}")
```

B6. Predictions on new text — Output

```
[Roll No: 24BAD047 | Jeevanantham K] Predictions on new text
POSITIVE (98.21%) <- An outstanding film with brilliant acting and a gripping storyline.
POSITIVE (98.44%) <- I loved every minute of it. Highly recommended!
NEGATIVE (98.46%) <- The plot was boring, the acting was terrible and I wasted my time.
NEGATIVE (98.17%) <- Worst movie I have seen this year. Absolutely awful.
NEGATIVE (98.10%) <- It was okay, some parts were good but the ending was disappointing.
```

B7. Saving the model and tokenizer — Code

```python
# Save fine-tuned model and tokenizer
SAVE_DIR = "./bert_imdb_finetuned"
trainer.save_model(SAVE_DIR)
tokenizer.save_pretrained(SAVE_DIR)
print(f"[Roll No: {ROLL_NO}] Saved to {SAVE_DIR}:", os.listdir(SAVE_DIR))

# Reload test (verifies the saved model works)
reloaded = AutoModelForSequenceClassification.from_pretrained(SAVE_DIR).to(device).eval()
re_tok = AutoTokenizer.from_pretrained(SAVE_DIR)
enc = re_tok("A wonderful, touching movie.", return_tensors="pt").to(device)
print("Reloaded model prediction:", reloaded.config.id2label[int(reloaded(**enc).logits.argmax(-1))])
```

B7. Saving the model and tokenizer — Output

```
[Roll No: 24BAD047] Saved to ./bert_imdb_finetuned: ['model.safetensors', 'config.json',
'training_args.bin', 'tokenizer.json', 'tokenizer_config.json']
Reloaded model prediction: POSITIVE
```

PART C: IMPLEMENTING A SEQUENCE-TO-SEQUENCE APPLICATION USING TRANSFORMERS

Perform the following tasks:

- Load a pretrained sequence-to-sequence Transformer model, such as T5-small.
- Load the corresponding tokenizer.
- Prepare a sample text input for the selected text generation task.
- Preprocess the input according to the model's requirements.
- Generate output text using the pretrained model.
- Apply appropriate generation parameters, such as maximum output length and beam search, where applicable.
- Display the input text and the generated output.
- Experiment with different input texts and generation settings.
- Observe how changes in the input and generation parameters affect the output.
- Save the generated outputs for further analysis.

C1. Loading T5-small — Code

```python
from transformers import AutoModelForSeq2SeqLM

print(f"[Roll No: {ROLL_NO} | {STUDENT_NAME}] PART C - Loading T5-small")
t5_name = "google-t5/t5-small"
t5_tokenizer = AutoTokenizer.from_pretrained(t5_name)
t5_model = AutoModelForSeq2SeqLM.from_pretrained(t5_name).to(device).eval()
print("T5 parameters: {:,}".format(sum(p.numel() for p in t5_model.parameters())))
```

C1. Loading T5-small — Output

```
[Roll No: 24BAD047 | Jeevanantham K] PART C - Loading T5-small
T5 parameters: 60,506,624
```

C2. Generation function and summarization — Code

```python
def t5_generate(text, prefix="summarize: ", max_new_tokens=60, min_new_tokens=10, num_beams=4,
                do_sample=False, temperature=1.0, top_p=1.0, no_repeat_ngram_size=3, length_penalty=1.0):
    inputs = t5_tokenizer(prefix + text, return_tensors="pt", max_length=512, truncation=True).to(device)
    kwargs = dict(max_new_tokens=max_new_tokens, min_new_tokens=min_new_tokens,
                  no_repeat_ngram_size=no_repeat_ngram_size, early_stopping=True)
    if do_sample:
        kwargs.update(do_sample=True, temperature=temperature, top_p=top_p, num_beams=1)
    else:
        kwargs.update(num_beams=num_beams, length_penalty=length_penalty)
    with torch.no_grad():
        out = t5_model.generate(**inputs, **kwargs)
    return t5_tokenizer.decode(out[0], skip_special_tokens=True)

article_1 = (
    "Artificial intelligence is transforming healthcare by enabling faster and more accurate diagnosis. "
    "Machine learning models can analyse medical images such as X-rays and MRI scans to detect diseases like "
    "cancer at early stages. Hospitals are also using AI to predict patient admissions, optimise staff schedules "
    "and reduce waiting times. However, experts warn that issues such as data privacy, algorithmic bias and a "
    "lack of transparency must be addressed before AI can be widely trusted in clinical practice."
)
article_2 = (
    "The Indian Space Research Organisation launched a new earth observation satellite from Sriharikota on "
    "Sunday. The satellite carries high resolution cameras that will help monitor agriculture, forests and "
    "water resources across the country. Scientists said the mission will also support disaster management by "
    "providing timely images of floods and cyclones. The launch vehicle placed the satellite into its intended "
    "orbit about eighteen minutes after lift-off."
)
article_3 = (
    "Regular exercise has many benefits for both the body and the mind. Studies show that thirty minutes of "
    "moderate activity each day lowers the risk of heart disease, improves sleep and reduces symptoms of anxiety "
    "and depression. Experts recommend combining aerobic exercise, strength training and stretching for the best "
    "results, and suggest that beginners start slowly and increase intensity gradually."
)
articles = {"Healthcare AI": article_1, "ISRO satellite": article_2, "Exercise": article_3}

print(f"[Roll No: {ROLL_NO} | {STUDENT_NAME}] T5 summarization (beam search, 4 beams)\n")
for name, txt in articles.items():
    print(f"INPUT [{name}] ({len(txt.split())} words):\n{txt}\n")
```

```python
    print("OUTPUT:", t5_generate(txt), "\n" + "-"*90)
```

C2. Generation function and summarization — Output

```
[Roll No: 24BAD047 | Jeevanantham K] T5 summarization (beam search, 4 beams)

INPUT [Healthcare AI] (77 words):
Artificial intelligence is transforming healthcare by enabling faster and more accurate diagnosis.
Machine learning models can analyse medical images such as X-rays and MRI scans to detect diseases like
cancer at early stages. Hospitals are also using AI to predict patient admissions, optimise staff
schedules and reduce waiting times. However, experts warn that issues such as data privacy, algorithmic
bias and a lack of transparency must be addressed before AI can be widely trusted in clinical practice.

OUTPUT: machine learning models can analyse medical images such as X-rays and MRI scans to detect
diseases like cancer at early stages. hospitals are also using AI to predict patient admissions,
optimise staff schedules and reduce waiting times. experts warn issues such as data privacy, algorithmic

------------------------------------------------------------------------------------------
INPUT [ISRO satellite] (65 words):
The Indian Space Research Organisation launched a new earth observation satellite from Sriharikota on
Sunday. The satellite carries high resolution cameras that will help monitor agriculture, forests and
water resources across the country. Scientists said the mission will also support disaster management by
 providing timely images of floods and cyclones. The launch vehicle placed the satellite into its
intended orbit about eighteen minutes after lift-off.

OUTPUT: the satellite carries high resolution cameras that will monitor agriculture, forests and water
resources across the country. the mission will also support disaster management by providing timely
images of floods and cyclones.
------------------------------------------------------------------------------------------
INPUT [Exercise] (60 words):
Regular exercise has many benefits for both the body and the mind. Studies show that thirty minutes of
moderate activity each day lowers the risk of heart disease, improves sleep and reduces symptoms of
anxiety and depression. Experts recommend combining aerobic exercise, strength training and stretching
for the best results, and suggest that beginners start slowly and increase intensity gradually.

OUTPUT: thirty minutes of moderate activity each day lowers the risk of heart disease. experts recommend
 combining aerobic exercise, strength training and stretching for the best results.
------------------------------------------------------------------------------------------
```

C3. Effect of generation parameters — Code

```python
# Experiment with generation parameters on article_1
print(f"[Roll No: {ROLL_NO}] Effect of generation parameters\n")
settings = {
    "Greedy (beams=1), max 30":      dict(num_beams=1, max_new_tokens=30),
    "Beam=4, max 30":                dict(num_beams=4, max_new_tokens=30),
    "Beam=4, max 60":                dict(num_beams=4, max_new_tokens=60),
    "Beam=4, max 100, min 40":       dict(num_beams=4, max_new_tokens=100, min_new_tokens=40),
    "Beam=8, max 60":                dict(num_beams=8, max_new_tokens=60),
    "Sampling T=0.7, top_p=0.9":     dict(do_sample=True, temperature=0.7, top_p=0.9, max_new_tokens=60),
    "Sampling T=1.5, top_p=1.0":     dict(do_sample=True, temperature=1.5, top_p=1.0, max_new_tokens=60),
}
torch.manual_seed(SEED)
exp_rows = []
for name, kw in settings.items():
    out = t5_generate(article_1, **kw)
    exp_rows.append({"Setting": name, "Output": out, "Words": len(out.split())})
    print(f"[{name}] ({len(out.split())} words)\n  {out}\n")
exp_df = pd.DataFrame(exp_rows)
```

C3. Effect of generation parameters — Output

```
[Roll No: 24BAD047] Effect of generation parameters

[Greedy (beams=1), max 30] (25 words)
  artificial intelligence can analyse medical images to detect diseases like cancer. hospitals also
using AI to predict patient admissions, optimise staff schedules and reduce waiting

[Beam=4, max 30] (22 words)
  machine learning models can analyse medical images such as X-rays and MRI scans to detect diseases
like cancer at early stages. hospitals

[Beam=4, max 60] (45 words)
  machine learning models can analyse medical images such as X-rays and MRI scans to detect diseases
like cancer at early stages. hospitals are also using AI to predict patient admissions, optimise staff
schedules and reduce waiting times. experts warn issues such as data privacy, algorithmic

[Beam=4, max 100, min 40] (62 words)
  machine learning models can analyse medical images such as X-rays and MRI scans to detect diseases
like cancer at early stages. hospitals are also using AI to predict patient admissions, optimise staff
schedules and reduce waiting times. experts warn issues such as data privacy, algorithmic bias and lack
of transparency must be addressed before AI can be widely trusted in clinical practice.

[Beam=8, max 60] (45 words)
  machine learning models can analyse medical images such as X-rays and MRI scans to detect diseases
like cancer at early stages. hospitals are also using AI to predict patient admissions, optimise staff
schedules and reduce waiting times. experts warn issues such as data privacy, algorithmic

[Sampling T=0.7, top_p=0.9] (26 words)
  AI can analyse medical images such as X-rays and MRI scans. hospitals also using AI to predict patient
 admissions, optimise staff schedules and reduce waiting times.

[Sampling T=1.5, top_p=1.0] (50 words)
  artificial intelligence can improve healthcare by offering quicker diagnostic. algorithms designed to
predict patients at young stage of a diagnosis. but experts warn security could help to correct mistakes
 and delays for long oax. scientists from the company argue with the right practices to better
understand the effectiveness of artificial
```

C4. Translation and other inputs — Code

```python
# Other tasks with the same model: translation and different inputs
tasks = [
    ("translate English to German: ", "The weather is very nice today and we are going to the park."),
    ("translate English to French: ", "I would like to book a table for two people this evening."),
    ("summarize: ", "Transformers use self attention to model relationships between all words in a sentence. "
                    "This allows them to capture long range dependencies and to be trained in parallel, which "
                    "makes them much faster to train than recurrent networks on large datasets."),
]
print(f"[Roll No: {ROLL_NO} | {STUDENT_NAME}] Other T5 tasks\n")
task_rows = []
for prefix, txt in tasks:
    out = t5_generate(txt, prefix=prefix, max_new_tokens=60, min_new_tokens=1)
    task_rows.append({"Task": prefix.strip(), "Input": txt, "Output": out})
    print(f"{prefix}\n  IN : {txt}\n  OUT: {out}\n")
```

C4. Translation and other inputs — Output

```
[Roll No: 24BAD047 | Jeevanantham K] Other T5 tasks

translate English to German:
  IN : The weather is very nice today and we are going to the park.
  OUT: Das Wetter ist heute sehr schön und wir gehen zum Park.

translate English to French:
  IN : I would like to book a table for two people this evening.
  OUT: Je voudrais réserver une table pour deux personnes ce soir.

summarize:
  IN : Transformers use self attention to model relationships between all words in a sentence. This
allows them to capture long range dependencies and to be trained in parallel, which makes them much
faster to train than recurrent networks on large datasets.
  OUT: Transformers use self attention to model relationships between words in a sentence. this allows
them to capture long range dependencies and to be trained in parallel.
```

C5. Saving generated outputs — Code

```python
# Save generated outputs for further analysis
rows = []
for name, txt in articles.items():
    rows.append({"Roll_No": ROLL_NO, "Task": "summarize", "Input_Name": name, "Setting": "beam=4,max=60",
                 "Input": txt, "Output": t5_generate(txt)})
for r in exp_rows:
    rows.append({"Roll_No": ROLL_NO, "Task": "summarize", "Input_Name": "Healthcare AI", "Setting": r["Setting"],
                 "Input": article_1, "Output": r["Output"]})
for r in task_rows:
    rows.append({"Roll_No": ROLL_NO, "Task": r["Task"], "Input_Name": "-", "Setting": "beam=4",
                 "Input": r["Input"], "Output": r["Output"]})
out_df = pd.DataFrame(rows)
out_df.to_csv("t5_generated_outputs.csv", index=False)
print(f"Saved {len(out_df)} generated outputs to t5_generated_outputs.csv")
out_df[["Task", "Setting", "Output"]].head(10)
```

C5. Saving generated outputs — Output

```
Saved 13 generated outputs to t5_generated_outputs.csv

 Unnamed: 0      Task                   Setting                                          Output
          0 summarize             beam=4,max=60 machine learning models can analyse medical im...
          1 summarize             beam=4,max=60 the satellite carries high resolution cameras ...
          2 summarize             beam=4,max=60 thirty minutes of moderate activity each day l...
          3 summarize  Greedy (beams=1), max 30 artificial intelligence can analyse medical im...
          4 summarize            Beam=4, max 30 machine learning models can analyse medical im...
          5 summarize            Beam=4, max 60 machine learning models can analyse medical im...
          6 summarize   Beam=4, max 100, min 40 machine learning models can analyse medical im...
          7 summarize             Beam=8, max 60 machine learning models can analyse medical im...
          8 summarize Sampling T=0.7, top_p=0.9 AI can analyse medical images such as X-rays a...
          9 summarize Sampling T=1.5, top_p=1.0 artificial intelligence can improve healthcare...
```

PART D: MODEL TESTING AND OUTPUT ANALYSIS

Perform the following tasks:

- Test the fine-tuned BERT model using sample positive and negative text inputs.
- Record the predicted class labels for the test inputs.
- Analyze the classification results using the selected evaluation metrics.
- Test the sequence-to-sequence model using different input texts.
- Compare the generated outputs for different inputs and generation settings.
- Examine the relevance, readability, and coherence of the generated text.
- Identify possible errors or limitations in the classification and generation outputs.
- Compare the applications of BERT-based classification and sequence-to-sequence text generation.
- Summarize the observations obtained from both models.

D1. BERT predictions on sample inputs — Code

```python
# D1. Test fine-tuned BERT with positive and negative inputs
test_inputs = {
    "POSITIVE": [
        "A masterpiece! The direction, music and performances were superb.",
        "Heartwarming and funny, I would happily watch it again.",
        "Great story and an amazing cast. One of the best films of the decade.",
        "Not bad at all, I was pleasantly surprised by how good it was.",       # mild negation
    ],
    "NEGATIVE": [
        "Dull, predictable and far too long. A complete waste of money.",
        "The dialogue was cringe-worthy and the acting was wooden.",
        "I walked out halfway through. Terrible script and poor editing.",
        "This movie is not good; I expected so much more.",                     # negation
    ],
}
rows = []
print(f"[Roll No: {ROLL_NO} | {STUDENT_NAME}] PART D - BERT predictions")
for true_lab, texts in test_inputs.items():
    for t, (pred, conf) in zip(texts, predict_sentiment(texts)):
        rows.append({"Text": t, "Expected": true_lab, "Predicted": pred, "Confidence": round(conf, 4),
                     "Correct": true_lab == pred})
bert_df = pd.DataFrame(rows)
pd.set_option("display.max_colwidth", 80)
display(bert_df)
print(f"\nManual-test accuracy: {bert_df['Correct'].mean():.2%} ({bert_df['Correct'].sum()}/{len(bert_df)})")
```

D1. BERT predictions on sample inputs — Output

```
[Roll No: 24BAD047 | Jeevanantham K] PART D - BERT predictions
```

Text	Expected	Predicted	Confidence	Correct
A masterpiece! The direction, music and performances were superb.	POSITIVE	POSITIVE	0.985	True
Heartwarming and funny, I would happily watch it again.	POSITIVE	POSITIVE	0.9672	True
Great story and an amazing cast. One of the best films of the decade.	POSITIVE	POSITIVE	0.9852	True
Not bad at all, I was pleasantly surprised by how good it was.	POSITIVE	POSITIVE	0.67	True
Dull, predictable and far too long. A complete waste of money.	NEGATIVE	NEGATIVE	0.9493	True
The dialogue was cringe-worthy and the acting was wooden.	NEGATIVE	NEGATIVE	0.9904	True
I walked out halfway through. Terrible script and poor editing.	NEGATIVE	NEGATIVE	0.969	True
This movie is not good; I expected so much more.	NEGATIVE	NEGATIVE	0.9596	True

```
Manual-test accuracy: 100.00% (8/8)
```

D2. Metrics and error analysis — Code

```python
# D2. Metrics summary and error analysis on the test set
metrics = {"Accuracy": acc, "Precision": prec, "Recall": rec, "F1-score": f1}
print(f"[Roll No: {ROLL_NO}] Evaluation metrics")
display(pd.DataFrame(metrics, index=["BERT (fine-tuned)"]).round(4))

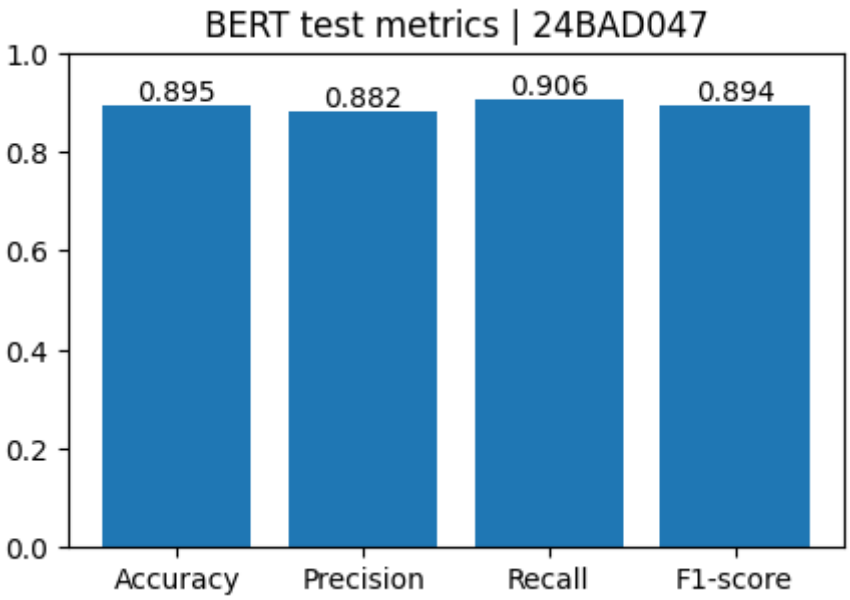
plt.figure(figsize=(5, 3.2))
plt.bar(metrics.keys(), metrics.values(), color="tab:blue")
plt.ylim(0, 1); plt.title(f"BERT test metrics | {ROLL_NO}")
for i, v in enumerate(metrics.values()): plt.text(i, v + .01, f"{v:.3f}", ha="center")
plt.show()

# Misclassified examples
test_texts = test_ds["text"]
wrong_idx = np.where(y_true != y_pred)[0]
print(f"Misclassified: {len(wrong_idx)} of {len(y_true)}")
for i in wrong_idx[:5]:
    print(f"\nTrue={id2label[int(y_true[i])]} | Pred={id2label[int(y_pred[i])]}\n{test_texts[i][:300]}...")
```

D2. Metrics and error analysis — Output

```
[Roll No: 24BAD047] Evaluation metrics

       Unnamed: 0  Accuracy  Precision  Recall  F1-score
BERT (fine-tuned)     0.895     0.8822  0.9057    0.8938
```




```
Misclassified: 105 of 1000

True=NEGATIVE | Pred=POSITIVE
Coming from Kiarostami, this art-house visual and sound exposition is a surprise. For a director known
for his narratives and keen observation of humans, especially children, this excursion into minimalist
cinematography begs for questions: Why did he do it? Was it to keep him busy during a vacation...

True=NEGATIVE | Pred=POSITIVE
"An astronaut (Michael Emmet) dies while returning from a mission and his body is recovered by the
military. The base where the dead astronaut is taken to becomes the scene of a bizarre invasion plan
from outer space. Alien embryos inside the dead astronaut resurrect the corpse and begin a terrifyin...

True=NEGATIVE | Pred=POSITIVE
I used to always love the bill because of its great script and characters, but lately i feel as though
it has turned into an emotional type of soap. If you look at promotional pictures/posters of the bill
now you will see either two of the officers hugging/kissing or something to do with friendships...

True=NEGATIVE | Pred=POSITIVE
A truly masterful piece of filmmaking. It managed to put me to sleep and to boggle my mind. So boring
that it induces sleep and yet so ludicrous that it made me wonder how stuff like this gets made. Avoid
at all costs. That is, unless you like taking invisible cranial punishment, in which case I hig...

True=POSITIVE | Pred=NEGATIVE
First, the positives: an excellent job at depicting urban landscapes to suit the mood of the film. Some
of the shots could be paintings by De Chirico. Sophie Marceau, beautiful.<br /><br />The negatives: the
stories are hard to believe. Unreal, uni-dimensional characters preen and posture 100% of th...
```

D3. T5 output analysis — Code

```python
# D3. Compare T5 outputs: relevance, readability, coherence
def overlap_ratio(src, out):
    s, o = set(src.lower().split()), out.lower().split()
    return sum(w in s for w in o) / max(len(o), 1)

rows = []
for name, txt in articles.items():
    out = t5_generate(txt)
    rows.append({"Input": name, "Input words": len(txt.split()), "Output words": len(out.split()),
                 "Compression": round(len(out.split()) / len(txt.split()), 2),
                 "Word overlap w/ source": round(overlap_ratio(txt, out), 2), "Output": out})
cmp_df = pd.DataFrame(rows)
print(f"[Roll No: {ROLL_NO} | {STUDENT_NAME}] T5 output analysis")
display(cmp_df)

print("Effect of generation settings (article_1):")
display(exp_df[["Setting", "Words", "Output"]])
```

D3. T5 output analysis — Output

```
[Roll No: 24BAD047 | Jeevanantham K] T5 output analysis
```

Input	Input words	Output words	Compressio	Word overlap w/ source	Output
Healthcare AI	77	45	0.58	1.0	machine learning models can analyse medical images such as X-rays and MRI sc...
ISRO satellite	65	32	0.49	1.0	the satellite carries high resolution cameras that will monitor agriculture,...
Exercise	60	26	0.43	0.92	thirty minutes of moderate activity each day lowers the risk of heart diseas...

```
Effect of generation settings (article_1):
```

Setting	Words	Output
Greedy (beams=1), max 30	25	artificial intelligence can analyse medical images to detect diseases like c...
Beam=4, max 30	22	machine learning models can analyse medical images such as X-rays and MRI sc...
Beam=4, max 60	45	machine learning models can analyse medical images such as X-rays and MRI sc...
Beam=4, max 100, min 40	62	machine learning models can analyse medical images such as X-rays and MRI sc...
Beam=8, max 60	45	machine learning models can analyse medical images such as X-rays and MRI sc...
Sampling T=0.7, top_p=0.9	26	AI can analyse medical images such as X-rays and MRI scans. hospitals also u...
Sampling T=1.5, top_p=1.0	50	artificial intelligence can improve healthcare by offering quicker diagnosti...

D4. BERT vs T5 comparison — Code

```
# D4. Comparison table: BERT classification vs. seq2seq generation
comparison = pd.DataFrame({
    "Aspect": ["Architecture", "Pretraining objective", "Output", "Task type", "Typical use cases",
               "Evaluation", "Main limitations"],
    "BERT classifier": ["Encoder-only", "Masked LM + Next Sentence Prediction", "One label from a fixed set + probability",
                        "Discriminative / understanding", "Sentiment, spam, topic, intent detection",
                        "Accuracy, precision, recall, F1", "Max 512 tokens; struggles with sarcasm/negation; fixed classes"],
    "T5 seq2seq": ["Encoder-decoder", "Span corruption (text-to-text)", "Free-form text of variable length",
                   "Generative", "Summarization, translation, QA, paraphrasing",
                   "ROUGE, BLEU, human judgement", "Can be repetitive/generic or hallucinate; t5-small has limited quality; output depends on decoding settings"],
})
display(comparison)
```

D4. BERT vs T5 comparison — Output

Aspect	BERT classifier	T5 seq2seq
Architecture	Encoder-only	Encoder-decoder
Pretraining objective	Masked LM + Next Sentence Prediction	Span corruption (text-to-text)
Output	One label from a fixed set + probability	Free-form text of variable length
Task type	Discriminative / understanding	Generative
Typical use cases	Sentiment, spam, topic, intent detection	Summarization, translation, QA, paraphrasing
Evaluation	Accuracy, precision, recall, F1	ROUGE, BLEU, human judgement
Main limitations	Max 512 tokens; struggles with sarcasm/negation; fixed classes	Can be repetitive/generic or hallucinate; t5-small has limited quality; outp...

EXPECTED OUTCOME

Students will understand the architecture and applications of Transformer models in Natural Language Processing. They will learn to fine-tune a pretrained BERT model for text classification, preprocess textual data, and evaluate classification performance using appropriate metrics. Students will also develop a simple sequence-to-sequence text generation application using a pretrained Transformer model and analyze the quality and relevance of the generated outputs.

OBSERVATIONS

• BERT classification: after 2 epochs on 4,000 training reviews the model reached Accuracy 0.895, Precision 0.882, Recall 0.906 and F1-score 0.894 on 1,000 test reviews (105 misclassified). Validation accuracy rose from 0.888 (epoch 1) to 0.895 (epoch 2).

• All 8 manual positive/negative test sentences, including ones with negation, were classified correctly. The mixed review "It was okay... the ending was disappointing" was predicted NEGATIVE

with about 98% confidence. Test-set errors mostly occurred on sarcastic, mixed-opinion and long reviews.
• T5-small generation: summaries were readable and relevant but mostly extractive (phrases copied from the source), in lowercase, and sometimes cut off mid-sentence when the maximum length was too small.
• Effect of parameters: increasing the maximum length from 30 to 60 to 100 (minimum 40) produced longer, more complete summaries (22, 45 and 62 words). Beam search (4 or 8 beams) was more fluent than greedy decoding. Sampling at temperature 0.7 stayed on topic, while temperature 1.5 produced incoherent, partly hallucinated text.
• Translation with T5-small gave correct German and French sentences for the sample inputs.
• Comparison: BERT (encoder-only) suits understanding and labelling text into fixed classes and is judged by accuracy/F1; T5 (encoder-decoder) suits generating new text and its output depends strongly on decoding settings.
• Limitations: BERT was trained on a small subset with inputs truncated to 256 tokens and can fail on sarcasm or mixed sentiment; T5-small is small and can repeat or omit facts and become incoherent at high temperature.

RESULT

A pretrained BERT model was fine-tuned for IMDb sentiment classification and achieved 89.5% accuracy (F1-score 0.894) on the test subset. A T5-small sequence-to-sequence model was used to generate summaries and translations, and the effect of input text and generation parameters on output quality was analysed.

POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What is the role of BERT in text classification?

Ans: BERT acts as a pretrained encoder that converts the input text into contextual embeddings; the [CLS] representation is passed through a classification layer that predicts the class label.

2. Why is tokenization required before providing text to a Transformer model?

Ans: Transformers work on numbers, not raw text. The tokenizer splits text into sub-word tokens, maps them to integer IDs, adds special tokens ([CLS], [SEP]) and builds attention masks, with padding/truncation giving a uniform input length.

3. What is the purpose of fine-tuning a pretrained BERT model?

Ans: To adapt BERT's general language knowledge to a specific task and dataset using much less data, time and compute, giving higher accuracy than training from scratch.

4. Mention any two-evaluation metrics used for text classification.

Ans: Accuracy and F1-score (precision and recall are also commonly used).

5. What is the role of the encoder and decoder in a sequence-to-sequence Transformer?

Ans: The encoder reads the whole input and builds contextual representations of it. The decoder generates the output tokens one at a time, using masked self-attention over tokens generated so far and cross-attention over the encoder output.

6. What is the difference between text classification and text generation?

Ans: Classification assigns one label from a fixed set to a given text, whereas generation produces new, variable-length text one token at a time.

7. How does the maximum output length affect text generation?

Ans: It limits the number of tokens generated. A very small value truncates the output and leaves it incomplete (seen with max 30 tokens), while a very large value gives longer output with possible repetition or irrelevant content and more computation.

8. Mention one real-world application of Transformer-based text classification and one application of text generation.

Ans: Text classification: sentiment analysis of customer reviews (or spam email detection). Text generation: automatic summarization of news articles (or machine translation / chatbots).

SUGGESTIONS FOR DATASETS & MODELS

Datasets:

- IMDb Movie Reviews – https://huggingface.co/datasets/stanfordnlp/imdb

- GLUE Benchmark – https://huggingface.co/datasets/nyu-mll/glue
- SST-2 Sentiment Classification – https://huggingface.co/datasets/nyu-mll/glue

Pretrained Models:

- BERT Base Uncased – https://huggingface.co/google-bert/bert-base-uncased
- T5-small – https://huggingface.co/google-t5/t5-small

ASSESSMENT

Description	Max Marks	Marks Awarded	
Pre Lab Exercise	**5**		
In Lab Exercise	**10**		
Post Lab Exercise	**5**		
Viva	**10**		
Total	**30**		
Faculty Signature			